Prototype : Counting people in Kumbh

How it works

A load-sensing strip that counts people crossing in each direction and ignores vehicles: the hardware, the firmware, the counting algorithm, the simulator that tests it, and the measured accuracy.

Repository	github.com/IkshitJ/Sensors
Firmware	v2, Arduino Mega 2560, 14 pads, 80 Hz
Accuracy (simulated)	94.6–99.7 % in total across 8 pedestrian scenarios; 93.3–98.8 % per direction
Date	5 November 2026

Contents

1. The system in one page 3

2. The physical installation 4

2.1 Bill of materials . . . 4

2.2 Wiring and scale . . . 4

3. Software architecture 5

4. The firmware: main loop and I/O 6

4.1 Start-up . . . 6

4.2 The 80 Hz frame . . . 6

4.3 Serial interface . . . 6

5. The counting engine, stage by stage 7

5.1 Why a delay line? . . . 7

5.2 Stage 1: baseline and median-of-3 . . . 8

5.3 Stage 2: vehicle detector . . . 8

5.4 Stage 3: auto-zero . . . 8

5.5 Stage 4: the wheel gate . . . 8

5.6 Stage 5: gait smoothing . . . 9

5.7 Stage 6a: arrivals (exact for individuals and small groups) . . . 9

5.8 Stage 6b: moving impulse (robust in crowds) . . . 10

5.9 Stage 7: sessions and the people estimate . . . 10

5.10 Stage 8: self-calibration . . . 11

5.11 A real crossing, end to end . . . 11

6. Calibration procedure 12

7. The simulator 13

7.1 The real firmware, unmodified . . . 13

7.2 The HX711 chip model . . . 13

7.3 The world model: people . . . 13

7.4 The world model: vehicles . . . 14

7.5 Ground truth . . . 14

7.6 Two ways to run it . . . 14

8. The web app 15

9. Accuracy 15

9.1 Vehicles . . . 16

9.2 Unseen seeds . . . 17

10. Limitations and next steps 17

Appendix: tuning constants 18

1. The system in one page

A strip of 14 load-sensing pads is laid flush into the road surface in the middle of a zebra crossing. It spans the full 7 m width of the road, so it covers both lanes. People walking along the lane step over it, and so do the wheels of every vehicle. An Arduino Mega reads the weight on every pad 80 times a second and decides, from the *shape* of that weight over time, how many people crossed and in which direction, while rejecting cars, buses, trucks, motorbikes, bicycles and auto-rickshaws.

The core problem. A pad only measures weight. A person, a group, a crowd, a bicycle wheel and a parked car all show up as "kilograms on a pad". The firmware has to turn that one number per pad into a head-count. It does this in three ideas:

- **Reject what is not a footstep.** Vehicle tyres load a pad far faster than any crowd can (more than 250 kg within 0.3 s). Light two-wheelers are caught by a *wheel gate*: their pulse is too short, it is flat where a footstep has a dip, or it comes as a front-and-rear wheel pair.
- **Measure throughput, not occupancy.** Instead of asking "how heavy is the strip right now?", the firmware accumulates how much weight *arrived* (arrivals) and how much moving weight × time passed over the strip (impulse). Dividing by one person's typical contribution gives the number of people.
- **Calibrate itself.** The per-person constants are learned continuously from people who clearly crossed alone, so the counter adapts to local body weights and walking speeds.

Direction comes for free. Pads 0–6 cover lane B (traffic moving ←) and pads 7–13 cover lane A (traffic moving →). The firmware runs one independent counting engine per lane, so each lane's count is that direction's count.

How it was tested. There is no hardware yet. Instead, the repository contains a simulator that compiles the *real, unmodified* Arduino sketch for a PC and feeds it with readings from 14 emulated HX711 ADC chips. Those chips sample a physics model of the road: people with realistic footstep forces, groups, crowds, people stopping on the strip, and seven vehicle types. The firmware is treated as a black box: its count is read only from what it prints on the serial port, and compared against the simulator's ground truth.

> **Result.** Across eight pedestrian scenarios, from 4 people per minute up to a 140-per-minute festival crowd, the total count is 94.6–99.7 % accurate and each direction 93.3–98.8 %. Bicycles are the weakest case (about 1 false count per 5 bicycles). These are simulated numbers; Section 10 lists what still needs validating on site.

2. The physical installation

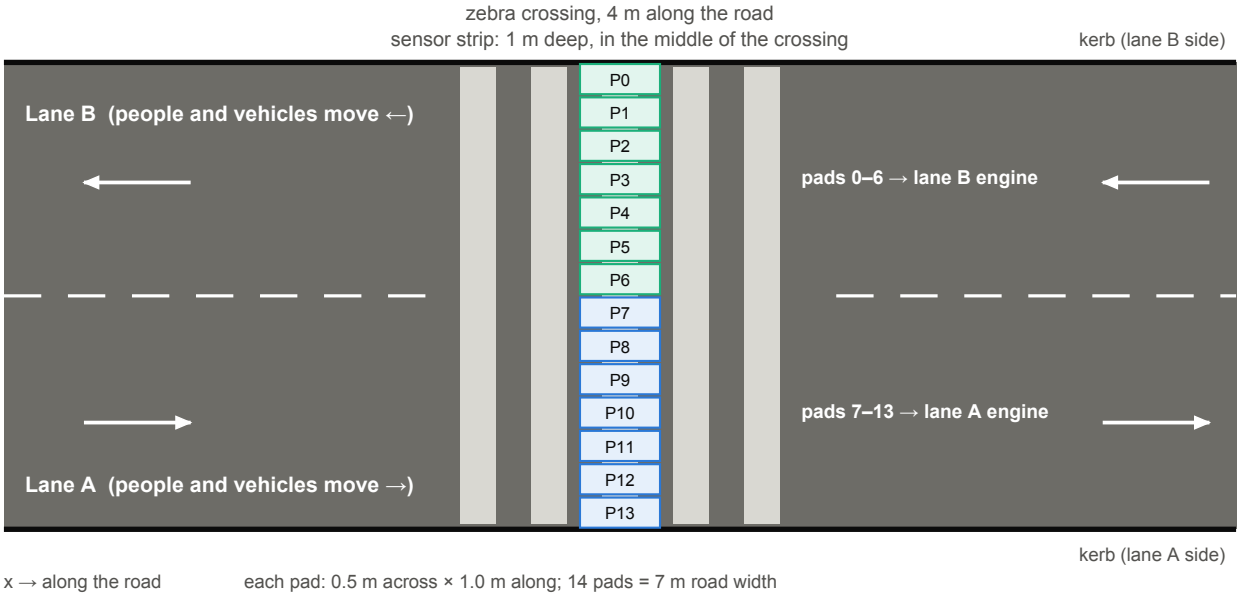


Figure 1. Top view of the road. The strip (14 pads) lies across both lanes in the middle of the zebra crossing. Pad 0 is at the lane-B kerb, pad 13 at the lane-A kerb.

Everyone travelling along the road crosses the strip's 1 m depth. A walking step is about 0.73 m long, so every pedestrian puts at least one foot, usually one or two, on at least one pad. Every vehicle also rolls its wheels over the strip, which is why rejecting vehicles is half of the job.

2.1 Bill of materials

Component	Qty	Notes
Arduino Mega 2560	1	About 4 KB of its 8 KB RAM used: two 7-pad engines plus I/O buffers.
HX711 load-cell amplifier	14	**RATE pin tied to VCC (80 samples/s).** Many breakout boards tie it to GND (10 samples/s): move the jumper or cut the trace. 10 SPS is far too slow to see a footstep's shape.
1000 kg shear-beam load cell, 2 mV/V	56	4 per pad wired as one bridge: 4000 kg capacity per pad, mechanical stop at 150 %.
Steel top plate 0.5 m × 1.0 m, ≥ 10 mm	14	Aluminium deforms under truck tyres.
Anti-slip rubber mat	1	Covers the strip, flush with the road.
IP65 enclosure, 5 V 3 A supply	1	

2.2 Wiring and scale

Pad *n* (0–13) has its HX711 DOUT on pin 22 + 2n and SCK on pin 23 + 2n, using pins 22–49. All HX711s share 5 V and GND. The nominal scale factor follows from the cells: 2 mV/V × 5 V = 10 mV at 4000 kg; at gain 128 one ADC count is 2.328 nV, so one kilogram is about **1073.7 counts**. Each pad's real factor is measured with the calibration sketch (Section 6).

Safety. The strip must be flush with the road surface. Any lip will damage the cells under traffic and trip pedestrians.

3. Software architecture

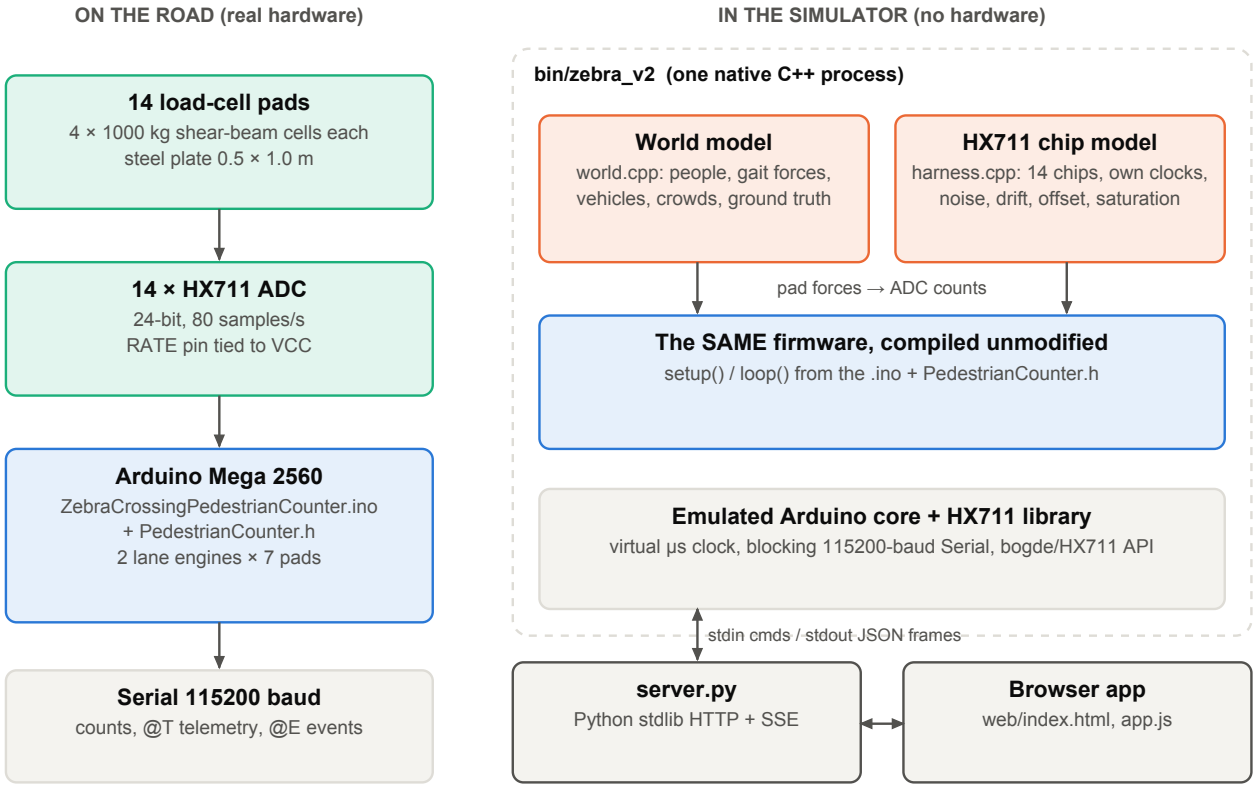


Figure 2. Left: the deployed system. Right: the simulator, which runs the same firmware source against emulated chips and a physics model, driven by a small web app.

The repository has three parts. The firmware is what gets uploaded to the Arduino. The simulator is everything needed to run that firmware on a laptop, and the web app is a live view and control panel for the simulator.

Path	What it is
firmware/ZebraCrossingPedestrianCounter/ ZebraCrossingPedestrianCounter.ino	The sketch: pin map, calibration factors, 80 Hz frame loop, two lane engines, Serial output and commands.
firmware/ZebraCrossingPedestrianCounter/ PedestrianCounter.h	The counting engine: pure computation, no I/O, no heap. All the logic of Section 5.
firmware/HX711_Calibration/	Interactive per-pad calibration sketch for all 14 pads.
simulation/src/world.*	Physics: pedestrians and their gait forces, groups, crowds, vehicles, ground truth.
simulation/src/harness.cpp	Runs setup()/loop(), emulates 14 HX711 chips, parses the firmware's Serial output, bench and interactive modes.
simulation/arduino/	Emulated Arduino core (virtual clock, Serial) and the HX711 library API.
simulation/server.py	Web server (Python standard library only): builds the binaries, runs one simulator, streams frames.
simulation/bench.py	Headless accuracy benchmark across scenarios and seeds, on every CPU core.
simulation/web/	Browser UI: road animation, PEOPLE CROSSED counter, Serial Monitor, controls, benchmark view.
legacy/	The original v1 sketches and README, kept for comparison.
Dockerfile	Container recipe for hosting the simulator web app (e.g. on Render).

4. The firmware: main loop and I/O

The sketch `ZebraCrossingPedestrianCounter.ino` owns the hardware. It creates 14 HX711 objects and two `PedestrianCounter` engines: `laneB` for pads 0–6 and `laneA` for pads 7–13. Each engine is compiled with `PC_MAX_PADS = 7` to keep its RAM low.

4.1 Start-up

- Open Serial at 115200 baud and print a banner.
- Begin each HX711 on its pin pair and apply its calibration factor (counts per kg).
- Initialise both engines with 7 pads each, then **tare**: average 20 readings per pad as its zero, and clear all signal history. The strip must be empty at power-up.

4.2 The 80 Hz frame

The HX711 at 80 samples/s produces a new reading every 12.5 ms, so the firmware works in fixed 12.5 ms frames. `loop()` runs as fast as it can and starts a new frame whenever 12.5 ms have passed since the last one (it resynchronises if it ever falls more than four frames behind):

```
if (micros() - lastFrameUs >= 12500) {
  readPads();                                  // 14 HX711s; skip chips not ready yet
  closedB = laneB.processFrame(&padKg[0], &padOverload[0]);
  closedA = laneA.processFrame(&padKg[7], &padOverload[7]);
  record worst frame time;                     // CPU budget check, shown by 's'
  if (closedB) printEvent('B', laneB);
  if (closedA) printEvent('A', laneA);
}
every 250 ms: print @T telemetry   |   every 1 s (if on): debug line   |   handleSerial()
```

Unsynchronised chips. The 14 HX711s each run their own conversion clock. `readPads()` reads only chips whose data is ready and keeps the previous value for the rest, so a frame never waits. A raw value at the 24-bit limit sets the pad's *overload* flag, which the vehicle detector treats as a tyre.

4.3 Serial interface

Key	Action
r	Reset the counts to zero.
t	Tare all pads (keep the strip clear).
s	Status: people crossed, per lane, vehicle events, transients rejected, learned V1/I1, worst frame time.
c	Calibration factors, offsets and current auto-zero per pad.
d	Human-readable pad debug line on/off (shows VEH on pads masked as vehicle).
m	Machine telemetry on/off.
h	Help.

Every crossing event prints a human line, and two machine-readable lines serve a data logger:

```
Lane A (->): +2 (est 1.87)   PEOPLE CROSSED: 154  [-> 83 | <- 71]

@T,<ms>,<total>,<live>,<vehicle active>,<vehicle pad mask>,<lane A>,<lane B>,<pad0 kg>,...,<pad13 kg>
@E,<ms>,<A|B>,<people this event>,<estimate>,<total>
```

@T is printed every 250 ms. `live` is the total plus the estimate for any session still in progress. @E is printed once per closed session. The simulator reads exactly these lines to learn the firmware's count.

5. The counting engine, stage by stage

`PedestrianCounter::processFrame()` is called once per frame per lane with the 7 tared pad weights in kg. It is pure computation: no I/O, no dynamic memory, no `String`. Figure 3 shows its pipeline; the stages are explained below with the exact thresholds from the code.



Figure 3. Per-frame pipeline of one lane engine. Stages 1–5 work on the newest samples; stages 6–9 work on the signal that leaves the 0.8 s delay line, after vehicles and wheels have been erased from it.

5.1 Why a delay line?

Many decisions can only be made *after* an event has been seen in full. A tyre's load ramps up over a few frames before it crosses the 250 kg trigger; a wheel pulse can only be judged as "too short" once it has ended. If the counter had already consumed those samples, it would be too late. So every pad's signal is pushed through a **0.8 s ring buffer** (64 frames). Detectors watch the newest end and, when they recognise a vehicle or a wheel, **erase or mask the samples retroactively** while they are still inside the buffer. The counting stages only ever see the cleaned signal coming out of the old end.

To fit the Mega's RAM, the buffer stores each sample as a 16-bit integer in 0.1 kg steps (±3200 kg; anything heavier is a tyre and masked anyway): 64 × 7 × 2 bytes = 896 bytes per lane. Each frame also stores a 7-bit

mask of pads covered by a vehicle.

5.2 Stage 1: baseline and median-of-3

The engine subtracts its own per-pad baseline (maintained by auto-zero, stage 3) from each input, then takes the median of the last three samples. A single corrupted ADC reading (a bit error) is removed completely, while real load changes pass through with one frame of delay.

5.3 Stage 2: vehicle detector

Trigger. A tyre is recognised by how *fast* load arrives, not by how much there is. A pad that gains **more than 250 kg within 0.3 s** (24 frames) is under a tyre: a crowd cannot load a 0.5 m-wide pad that fast. To ignore lone glitches, the previous sample must also already be above 40 kg. A tyre on the border of two pads splits its load, so adjacent pairs are also checked: a pair sum above 260 kg that rose by more than 200 kg within 0.1 s is a tyre too. An ADC overload is always a tyre.

Masking only what is under the tyre. When a vehicle event starts, each pad's *reference* is set to its lowest level in the delay line, i.e. its load just before the vehicle. A pad is then *latched* while it stays more than 180 kg above that reference (or 230 kg for a pad pair), plus a 150 ms tail. Only latched pads are masked, so a pedestrian walking next to a car on the other pads is still counted. Masked pads output their last clean value (they are held, not zeroed).

Retroactive mask. When a pad is newly latched, the last 0.4 s (32 frames) of that pad in the delay line are also masked. That hides the tyre's ramp-up, which happened before the trigger fired and would otherwise look like a heavy footstep arriving.

Parked vehicles. If a latched pad's raw value stays within a 15 kg band for 3 s, the tyre is standing still. Its weight is then absorbed into the pad's zero, without moving the output, and the pad is released, so people stepping next to a parked car are counted again. When the car drives off the pad reads strongly negative, and auto-zero corrects it within 0.5 s (next stage).

5.4 Stage 3: auto-zero

Condition (newest sample)	Action	Why
Below −6 kg for 0.5 s	Re-zero the pad to the current value	A load that had been absorbed (parked tyre, debris) has left.
Within ±3 kg and no footstep in progress for 2 s	Track slowly: move the zero 0.4 % of the reading per frame (applied in batches every 32 frames)	Temperature drift of the cells and bridge.
Above 8 kg and steady within ±4 kg for 15 s	Absorb the load into the zero	Debris, a dropped bag, or someone standing still for a long time.

Every zero shift goes through one function, `shiftPad()`, which moves the baseline and every stored state of that pad (median history, delay line, filters, trackers) by the same amount. This keeps the whole pipeline consistent, so a re-zero never looks like a person arriving or leaving.

5.5 Stage 4: the wheel gate

Motorbikes, bicycles and auto-rickshaws are too light for the tyre detector: a bicycle wheel carries about 45 kg, less than a person. They are rejected by the **wheel gate**, which watches each pad's newest samples as a sequence of *episodes*: an episode starts when the pad exceeds 6 kg and ends after 50 ms below 3 kg. When an episode ends it is judged; if it is a wheel, its samples are erased from the delay line before the counter ever sees them.

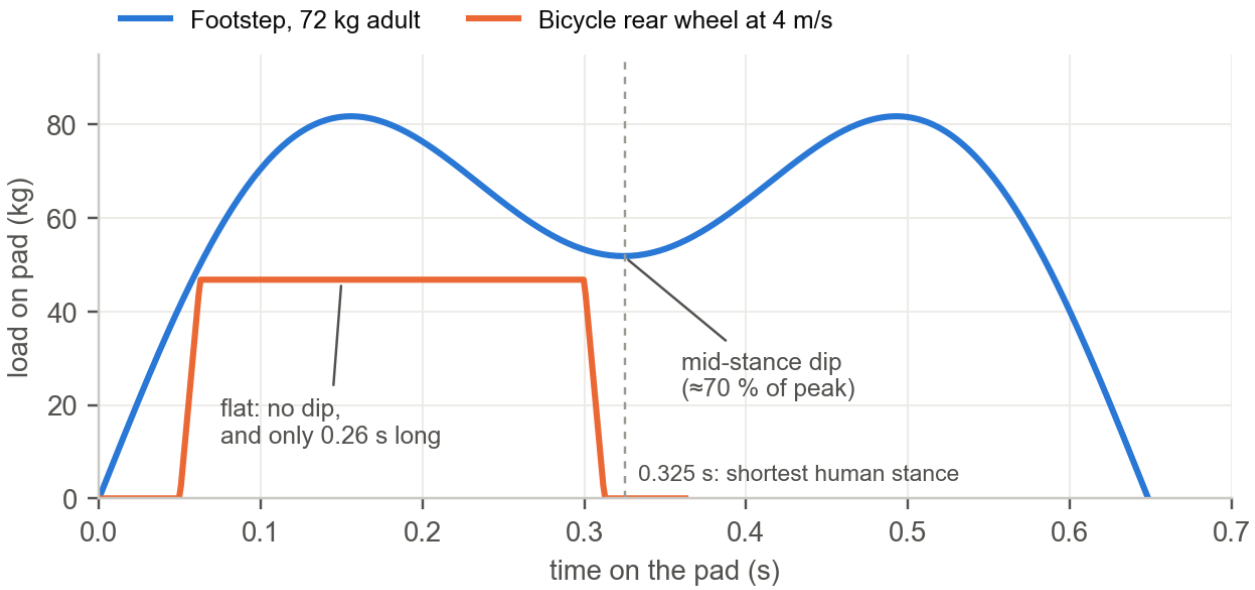


Figure 4. Load on one pad: a footstep from the simulator's gait model (blue) and a bicycle's rear wheel at 4 m/s (orange), both computed from simulation/src/world.cpp. The footstep is longer and has a dip; the wheel is short and flat.

A pulse shorter than **750 ms** is erased if any of these is true:

- **Too short:** it lasted less than **325 ms**, shorter than any human stance phase. A walking foot is on the ground for about 0.6–0.8 s.
- **No footstep dip:** a walking footstep has the classic *double hump*: heel strike, a mid-stance dip, then toe-off. The gate requires a valley between 40 % and 88 % of the peak followed by a recovery of at least 12 % of the peak. A rolling wheel's load is flat, so it never shows this.
- **Wheel pair:** it is shorter than 500 ms and another short pulse (also under 500 ms) started on the same or a neighbouring pad 75–700 ms earlier: a front and rear wheel one wheelbase apart. Both pulses are erased.

Pulses of 750 ms or longer are never erased: those are slow footsteps or someone standing.

5.6 Stage 5: gait smoothing

Leaving the delay line, each pad's signal passes two cascaded exponential low-pass filters ($\alpha = 0.10$ per frame, time constant about 0.12 s). This removes the ripple of the double hump and of body sway, so a single footstep becomes one smooth bump.

5.7 Stage 6a: arrivals (exact for individuals and small groups)

The first people measure answers: *how much weight arrived on the strip?* It accumulates only the **rising** part of the smoothed load, through a backlash operator with a 0.8 kg dead band so that noise and sway do not add up. One person adds about one body weight of arrivals, however many steps they take on the pad, because a foot that lifts and lands again on the same pad first falls and then rises.

The difficulty is pad borders. A foot often straddles two pads, splitting its weight. Summing the rises of single pads would count both halves at the wrong times; summing over the whole lane would let one person's arrival cancel another's departure. The engine uses **inclusion–exclusion over overlapping 2-pad windows**:

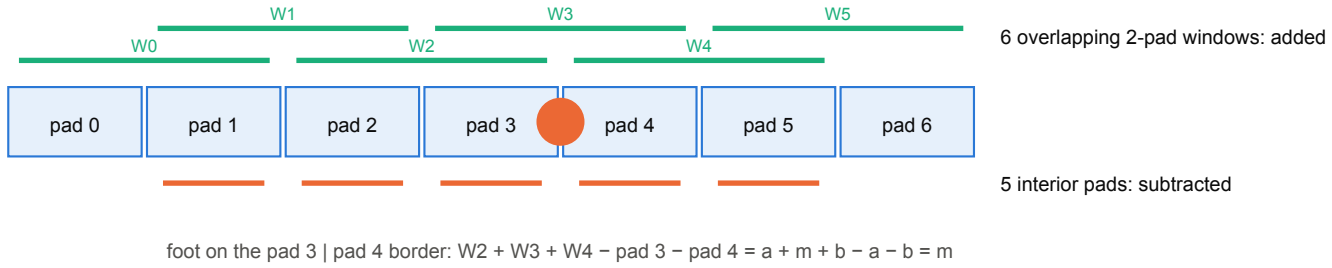


Figure 5. Arrivals = Σ rise(window of pads i, i+1) − Σ rise(interior pad i). Every position on the strip ends up counted exactly once.

A foot entirely on interior pad 3 rises in windows W2 and W3 (+2m) and in pad 3 (−m): net m. A foot split a / b across the pad 3–4 border shows a in W2, m in W3, b in W4, minus a for pad 3 and b for pad 4: net m. Edge pads (0 and 6) sit in only one window and are not subtracted. So each person adds about one body weight regardless of where they step.

Limit: in a dense crowd, one person arriving on a pad while another leaves it partly cancel out, so arrivals undercount crowds. That is what the second measure is for.

5.8 Stage 6b: moving impulse (robust in crowds)

The second measure is **impulse**: load × time, summed over pads. A person walking over the 1 m strip delivers a fairly constant amount of weight × time. Unlike arrivals, impulse cannot cancel, so it keeps working in a packed crowd. Its weakness is people or wheels that *stand still*: they keep adding impulse while not crossing. So only **moving** load counts:

- **Fresh:** the pad saw a new footstep, i.e. the unsmoothed load rose more than 8 kg above a slowly rising valley tracker, within the last 1 s.
- **Not flat:** load above 15 kg that stays within a 6 kg band for 0.3 s is standing still. Walking load always ripples.
- **Minus standing load:** when a footstep arrives on a pad that already carried a standing person or a stopped wheel, that static part is subtracted, so only the newcomer's load is integrated.

Each frame adds Σ(moving load on fresh pads above 2 kg) × 12.5 ms to the session's impulse.

5.9 Stage 7: sessions and the people estimate

Activity on a lane is grouped into **sessions**. A session opens when any pad has more than 6 kg (or its smoothed load exceeds 2 kg) and closes after **250 ms** of quiet. During a session the engine accumulates arrivals *v* and impulse *imp*, and estimates the people in it from two learned constants: **V1**, the arrivals of one person (default 68 kg), and **I1**, the impulse of one person (default 51 kg·s).

```
nV = v / V1        nI = imp / I1        n = max(nV, nI)
n <= 0.8        -> estimate = nV                          (single walkers: arrivals are exact)
n >= 2.5        -> estimate = nI                          (crowds: impulse cannot cancel)
in between      -> estimate = nV + w * (nI - nV),  w = (n - 0.8) / 1.7
```

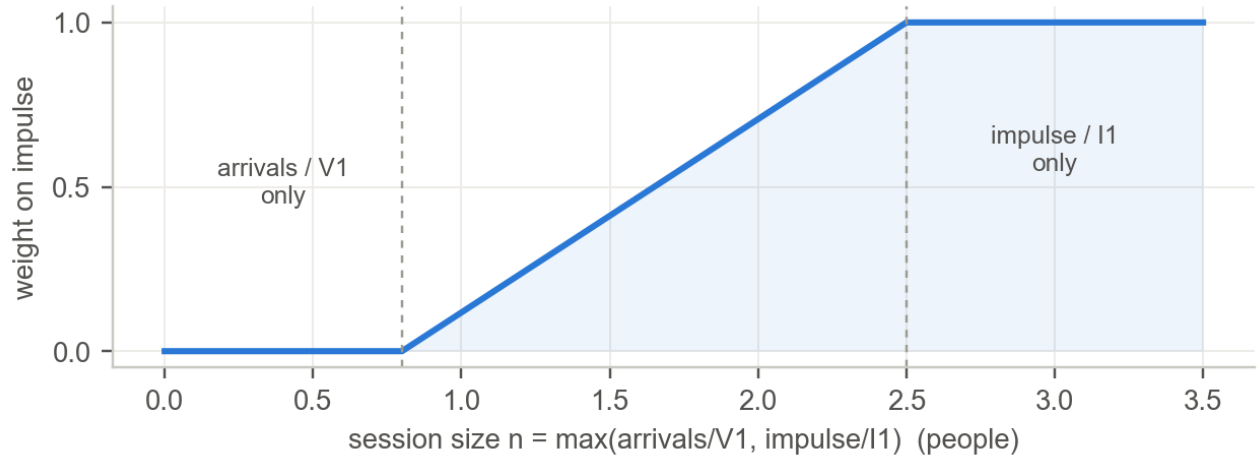


Figure 6. Weight given to the impulse estimate as the session grows.

Closing a session. Small sessions (estimate below 3.5 people) are rounded to the nearest whole person, with any estimate of at least 0.2 counting as at least one person; that is exact for singles and small groups. Large sessions keep **fractions**: the estimate is added to a running fractional accumulator and only whole people are moved to the total, so there is no rounding bias in a long crowd flow. Estimates below 0.2 are treated as noise.

Long crowds. A continuous crowd may never fall quiet for 250 ms. Whenever a session's estimate exceeds 20 people, 10 are committed to the total immediately and the session's accumulators are scaled down to match. The count therefore keeps moving during a festival crowd, and nothing is lost if the crowd thins out without a clean gap.

5.10 Stage 8: self-calibration

V1 and I1 depend on the local population (body weight) and walking speed. Every closed session that is *unmistakably one person* nudges them towards its measured values with a learning rate of 3 %. A session qualifies only if all of these hold:

- no vehicle was masked during it, and it was not a large crowd session;
- it touched one or two *adjacent* pads only;
- it lasted between 0.5 s and 2.25 s;
- its moving load never exceeded 130 kg;
- its estimate was between 0.5 and 1.6 people.

V1 is clamped to 40–110 kg and I1 to 25–110 kg·s, so a run of odd sessions cannot push them into nonsense. The `s` command shows the learned values and how many singles they came from.

5.11 A real crossing, end to end

Figure 7 is a recording from the running simulator: one adult walks along lane A and puts one foot on pad 12. The double hump is clearly visible (63 kg → 51 kg → 63 kg). The firmware printed `Lane A (->): +1 (est 0.77)`: the estimate of 0.77 people rounded to one.

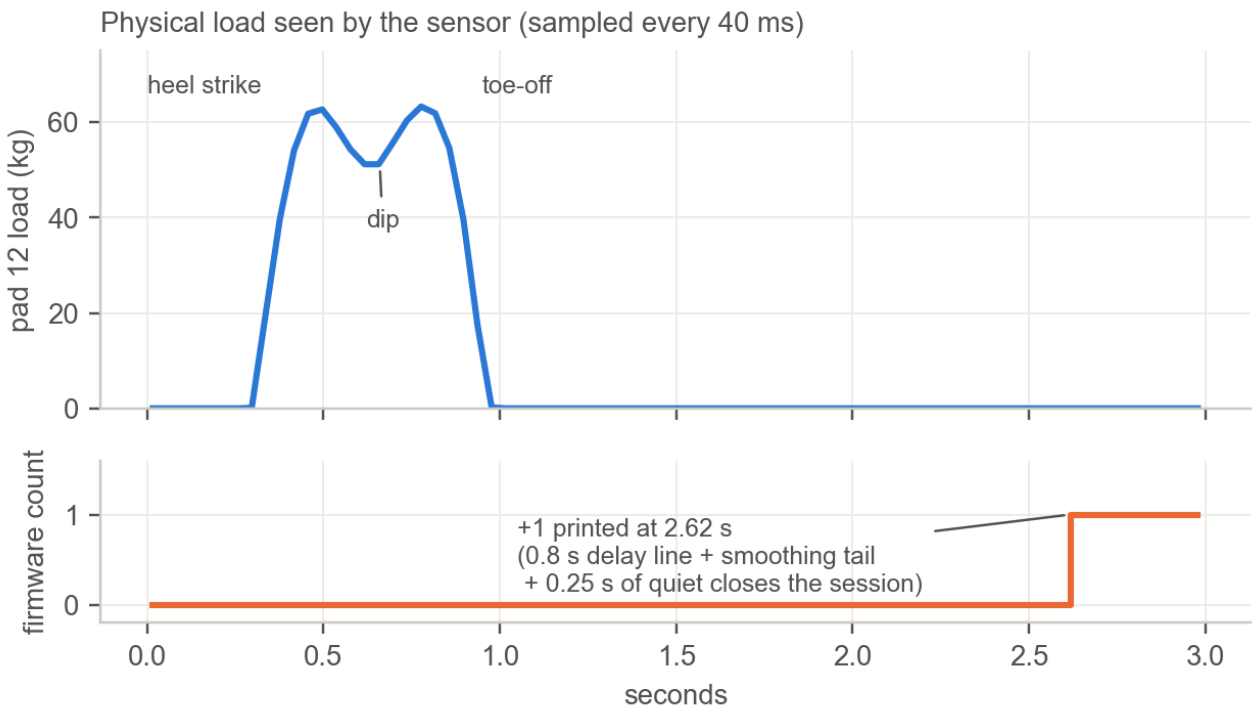


Figure 7. Top: pad 12 load as produced by the simulator. Bottom: the firmware's printed total. The count appears about 1.6 s after the foot lifts: 0.8 s through the delay line, the smoothing filters' tail, and 250 ms of quiet that closes the session.

The latency is deliberate: the counter only commits once it is sure the pulse was a person and not the start of a vehicle. For counting purposes, a delay of 1–2 s is irrelevant.

6. Calibration procedure

Each pad's four load cells and its HX711 have a slightly different sensitivity (about ±2 %). The calibration sketch `firmware/HX711_Calibration/HX711_Calibration.ino` measures every pad's factor in ADC counts per kg:

Step	Action
1	Upload the calibration sketch, open the Serial Monitor at 115200 baud with line ending set to Newline.
2	With the pad empty, send `t` (tare).
3	Put a known weight of at least 20 kg on the pad and send `k20` (use the actual mass).
4	Send `n` for the next pad and repeat for all 14.
5	Send `a`, then paste the printed CALIBRATION_FACTORS array into ZebraCrossingPedestrianCounter.ino and upload it.

The whole procedure can be rehearsed in the simulator: choose *Calibration sketch* in the web app and use its *Calibration bench* panel to place weights on pads.

7. The simulator

The simulator answers one question: *if this exact firmware were installed on a real road, how accurately would it count?* Three design choices make that answer credible.

7.1 The real firmware, unmodified

`src/sketch_v2.cpp` is four lines: it includes the emulated `Arduino.h` and then the real `.ino` file. The firmware's `setup()` and `loop()` are compiled natively with clang++/g++ against:

- **An emulated Arduino core** with a virtual microsecond clock. `micros()` and `millis()` read simulated time, and every `loop()` pass costs 20 µs of it.
- **A realistic Serial port**: 115200 baud 8N1 with the AVR's 64-byte transmit buffer. Once the buffer is full, `Serial.print` blocks and burns virtual time exactly as on the real board.
- **The HX711 library** with the same API as the widely used bogde/HX711, charging about 60 µs per read for bit-banging 25 clock pulses.

The firmware is a black box: the harness learns its count only by parsing the @T and @E lines it prints.

7.2 The HX711 chip model

Effect	How it is modelled
Conversion timing	Each of the 14 chips has its own 80 SPS clock with a random phase; data-ready is per chip, as on the real board.
Sigma-delta averaging	Each reading averages the physical load at 4 points across its 12.5 ms conversion window.
Scale and tolerance	1073.7 counts/kg nominal, ±2 % per pad (random, normal).
Bridge offset	Random offset of ±60,000 counts per chip (removed by tare).
Noise	Mechanical 0.3 kg RMS plus ground vibration from heavy traffic nearby; electrical 39 counts RMS.
Temperature drift	Slow sine of 0.3–1.5 kg with a 15–40 minute period, per pad.
Bit errors	One reading in 50,000 has bit 19 flipped (≈ +490 kg spike).
Limits	24-bit saturation; the plate bottoms out mechanically at 150 % of 4000 kg.

7.3 The world model: people

Every pedestrian gets a sequence of footprints along their lane, with ±9 cm lateral foot placement, small random scatter and a slightly drifting path. Each footstep's vertical force follows the standard **double-hump ground reaction force**:

```
F(s) = mass × k × ( A·sin(π s) + B·sin(3π s) ),   s = 0..1 over the stance phase
A ≈ 1.21, B ≈ 0.46 (randomised per person); k normalises every step to deliver
exactly body weight × step period of impulse. Runners: pure sin(π s), with a flight phase.
```

The centre of pressure rolls 20 cm from heel to toe during the stance. The loaded part of the sole is about 12 cm × 9 cm, so a foot on the plate edge shares its load with the road, and a foot on a pad border splits it between pads.

Type	Mass (kg)	Speed (m/s)	Cadence (steps/s)	Share
Adult	72 ± 13 (+2–10 kg bag for 25 %)	1.35 ± 0.17	1.85	the rest
Teen	54 ± 9	1.40 ± 0.18	1.90	random
Child	24 ± 7	1.15 ± 0.20	2.30	6 %

Type	Mass (kg)	Speed (m/s)	Cadence (steps/s)	Share
Elderly	68 ± 12	0.95 ± 0.12	1.65	6 %
Runner	70 ± 11	3.0 ± 0.4	2.80	2 %

- **Groups** of 2 (55 %), 3 (30 %) or 4 (15 %) walk side by side 0.48–0.65 m apart, at a shared speed, not perfectly abreast.
- **Crowds slow down** following Weidmann's fundamental diagram: walking speed falls with the density of people within 5 m of the strip.
- **Waiting:** some people stop on the strip for 3–25 s, standing on both feet with slight sway.
- **Avoidance:** people step around a vehicle that is standing on the crossing when they arrive.

7.4 The world model: vehicles

Type	Mass (kg)	Axles	Tyre patch	Share of traffic
Car	1150–1800 + 70–300 load	2 (57 / 43 %)	16 cm × 20 cm	42 %
Motorbike	120–180 + 60–150	2, single track	9 cm × 12 cm	24 %
Auto-rickshaw	330–400 + 70–350	1 front wheel, 2 rear	10 cm × 11 cm	10 %
SUV	1800–2500 + 70–400	2	18 cm × 24 cm	9 %
Bicycle	10–16 + 35–95 rider	2, single track	5 cm × 4 cm	6 %
Truck	7000–25000	3	22 cm × 50 cm	5 %
Bus	11000–16500	2	22 cm × 50 cm	4 %

Each axle's load sits on its tyre contact patches and is spread over the pads they cover. Rolling vehicles bounce on their suspension (±5 % at 1.6 Hz); stopped ones vibrate with the idling engine. Some vehicles stop with their front wheels on the strip for 2–20 s, then drive off.

7.5 Ground truth

A person counts as **crossed** at the moment their last foot leaves the strip, and is attributed to their lane. A vehicle counts as passed once its rear has cleared the strip. Accuracy is computed per run as **1 − |counted − true| / true**, in total and per lane.

7.6 Two ways to run it

- **Benchmark mode** (`zebra_v2 --bench --scenario dense --duration 600 --seed 1`): boots the firmware on an empty strip for 2 s, runs the scenario as fast as the CPU allows, stops spawning, lets everyone finish crossing, and prints one JSON line with truth and counts.
- **Interactive mode** (`--interactive`): runs in real time (adjustable 0.05×–50×), accepts commands on stdin (spawn people, groups or vehicles in a chosen lane, change rates, type into the Serial Monitor, place calibration weights) and writes a JSON frame every 40 ms with positions, pad loads, the firmware's count and new Serial output.

8. The web app

`server.py` uses only the Python standard library. At start-up it runs `make` to build both simulator binaries, starts one interactive simulator process, and serves the browser UI. A background thread reads the simulator's frames and fans them out to every connected browser.

Endpoint	Purpose
GET /	The app: web/index.html, style.css, app.js.
GET /api/stream	Server-Sent Events: one JSON frame every 40 ms (positions, pad loads, counts, Serial text).
POST /api/cmd	Text command passed to the simulator, e.g. `spawn group 3 A` or `serial r`.
POST /api/firmware	Restart with the main or the calibration sketch, a scenario and a seed.
POST /api/bench, GET /api/bench	Start the accuracy benchmark in the background; poll its progress and result.
GET /api/state	Which firmware and scenario are currently running.

In the browser you can watch people and vehicles move over the strip, see the **PEOPLE CROSSED** counter (total and → / ← split) next to the ground truth and the live accuracy, choose a scenario, change traffic rates and simulation speed, spawn people, groups and vehicles by hand, use the Serial Monitor exactly as on the real board, switch to the calibration sketch, and run the benchmark on every CPU core.

```
cd simulation
python3 server.py                    # builds, then serves http://localhost:8080
python3 bench.py                     # 9 scenarios x 3 seeds x 10 simulated minutes
python3 bench.py --seeds 5 --seed-start 100     # check on unseen random seeds
```

Hosting. The app needs a long-running process and a never-ending stream, so it cannot run on serverless platforms such as Vercel. The repository's `Dockerfile` installs g++ and make, builds the simulator into the image and starts the server on `0.0.0.0:$PORT`, so it deploys as-is to Render, Railway or Fly.io.

9. Accuracy

Each scenario was run with 3 random seeds × 10 simulated minutes. The firmware's count comes only from its Serial output. Figure 8 shows the total and per-direction accuracy.

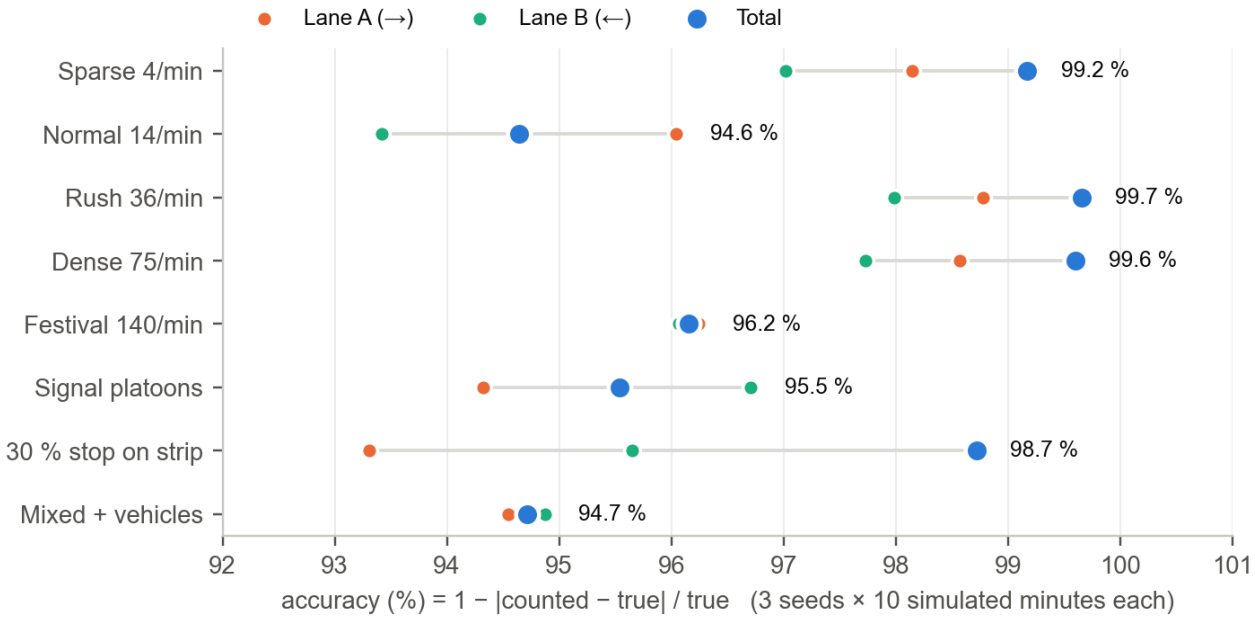


Figure 8. Accuracy by scenario (simulation/results.json). Total in blue; each direction as a smaller dot. Errors in the two lanes partly cancel in the total.

Scenario	Traffic	True	Counted	Total	→ A	← B
Sparse	4 people/min, alone	121	122	99.2 %	98.1 %	97.0 %
Normal	14/min, 25 % groups, 4 vehicles/min	597	565	94.6 %	96.0 %	93.4 %
Rush	36/min, 35 % groups, vehicles	1,744	1,750	99.7 %	98.8 %	98.0 %
Dense	75/min continuous, 40 % groups	3,736	3,751	99.6 %	98.6 %	97.7 %
Festival	140/min, 50 % groups	7,626	7,919	96.2 %	96.2 %	96.1 %
Signal	~45 people every 60 s + 6 vehicles/min	1,368	1,307	95.5 %	94.3 %	96.7 %
Waiting	30 % stop on the strip 3–25 s	469	463	98.7 %	93.3 %	95.7 %
Mixed	18/min + 10 vehicles/min, 20 % stop	851	806	94.7 %	94.5 %	94.9 %

9.1 Vehicles

In the vehicles-only scenario (24 vehicles/min, all types, 8 % stopping on the strip) the firmware produced **43 false people from 728 vehicles**. Figure 9 breaks this down by vehicle type.

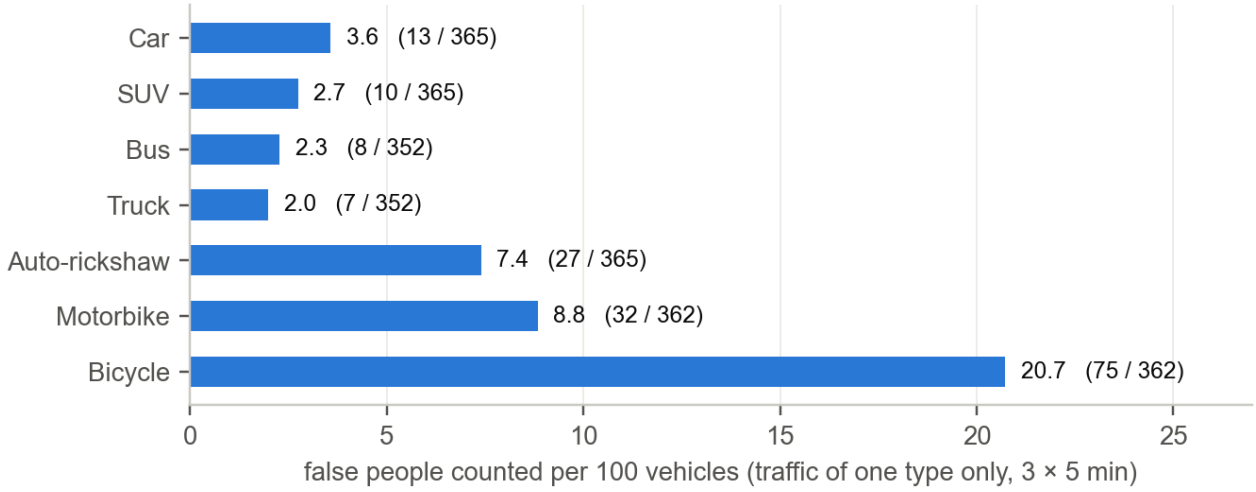


Figure 9. False counts per 100 vehicles, with traffic of a single type.

Heavy vehicles are caught reliably by the tyre detector. All of the car false counts come from cars that stop on the strip and drive off; with no stops, cars produced none. Light two-wheelers are the weak point: below about 10 km/h their wheels look like slow footsteps.

9.2 Unseen seeds

All thresholds were tuned on seeds 1–3. On five unseen seeds (100–104), the total scored 94.4–99.1 % per scenario and each direction 93.4–100 %. Traffic produced 64 false counts from 1,202 vehicles. The worst single 10-minute run was 89.6 %.

10. Limitations and next steps

- **These are simulated numbers.** Before relying on the counts, validate on site against a manual count. Self-calibration absorbs most differences in local population and walking speed, but the physics model is still a model.
- **Bicycles and slow two-wheelers** remain the weakest case: about 1 false count per 5 bicycles and 1 per 11 motorbikes. Separating them reliably would need a second row of pads (to measure speed) or another sensor.
- **Stopped vehicles:** a car that stops on the strip and drives off adds about 0.4 false counts on average.
- **Runners:** a running footstep lasts about 0.25 s, so the wheel gate removes most runners (about 2 % of people in the simulated population).
- **Not yet compiled for the AVR.** The firmware compiles cleanly as strict C++11 (what avr-gcc uses) and avoids String, heap allocation and 16-bit int overflow, but it has not been built with the AVR toolchain. Compile it once in the Arduino IDE, then send `s` to check the **worst frame time**: two engines × 7 pads of float maths plus 14 ADC reads must fit in 12.5 ms on a 16 MHz AVR. The simulator only charges for the ADC reads, so its reported 840 µs is not representative. If the budget is tight, a Teensy 4.x or an ESP32 runs the same code with a large margin.

Appendix: tuning constants

All constants live in the `pc` namespace at the top of `PedestrianCounter.h`. One frame is 12.5 ms.

Constant	Value	Meaning
DELAY_FRAMES	64 (0.8 s)	Length of the look-ahead delay line.
ON_KG / OFF_KG / OFF_FRAMES	6 / 3 kg / 4 (50 ms)	Pad episode start, end threshold and hold-off.
SHORT_FRAMES	26 (325 ms)	Shorter pulses are wheels.
GAIT_CHECK_FRAMES	60 (750 ms)	Pulses shorter than this must show a footstep dip.
GAIT_DIP_LO / HI / RECOVER	0.40 / 0.88 / 0.12	Dip window and recovery, relative to the peak.
PAIR_MAX_FRAMES	40 (500 ms)	Pulses shorter than this are checked for a wheel pair.
WHEEL_MIN_LAG / MAX_LAG	6 / 56 (75–700 ms)	Front-to-rear wheel delay window.
SMOOTH_ALPHA	0.10	Two cascaded EMAs, $\tau \approx 0.12$ s.
BACKLASH_KG	0.8 kg	Dead band of the arrivals measure.
SESSION_GAP_FRAMES	20 (250 ms)	Quiet time that closes a session.
ACTIVE_KG	2 kg	Smoothed load that keeps a session open.
DEFAULT_V1 (range)	68 kg (40–110)	Arrivals per person, learned.
DEFAULT_I1 (range)	51 kg·s (25–110)	Impulse per person, learned.
LEARN_ALPHA	0.03	Self-calibration learning rate.
BLEND_LO / BLEND_HI	0.8 / 2.5 people	Arrivals → impulse blend range.
FRESH_RISE_KG / FRESH_FRAMES	8 kg / 80 (1 s)	A new footstep keeps a pad's load moving for 1 s.
FLAT_BAND_KG / FLAT_MIN_KG / FLAT_FRAMES	6 kg / 15 kg / 24 (0.3 s)	Load that is standing still.
ROUND_LIMIT	3.5 people	Larger sessions keep fractions.
MIN_PERSON	0.2	Smaller sessions are noise.
LARGE_COMMIT	20 people	Crowd sessions commit in chunks of 10.
VEH_TRIGGER_KG / VEH_RISE_FRAMES	250 kg / 24 (0.3 s)	Tyre trigger on one pad.
VEH_PAIR_TRIGGER_KG / RISE_KG / RISE_FRAMES	260 / 200 kg / 8 (0.1 s)	Tyre trigger on a pad pair.
VEH_CONFIRM_KG	40 kg	Previous sample must agree (not a glitch).
VEH_TRACK_KG / VEH_TRACK_PAIR_KG	180 / 230 kg	Keep a pad latched while above its reference by this.
VEH_POST_FRAMES / VEH_RETRO_FRAMES	6 (150 ms) / 32 (0.4 s)	Tail after a tyre; retroactive mask.
PARK_FRAMES / PARK_BAND_KG	240 (3 s) / 15 kg	Stationary tyre is absorbed into the zero.
IDLE_FRAMES / IDLE_ALPHA	160 (2 s) / 0.004	Drift tracking while idle.
NEG_KG / NEG_FRAMES	−6 kg / 40 (0.5 s)	Re-zero after load leaves.
STATIC_MIN_KG / BAND_KG / FRAMES	8 / 4 kg / 1200 (15 s)	Absorb a long static load.